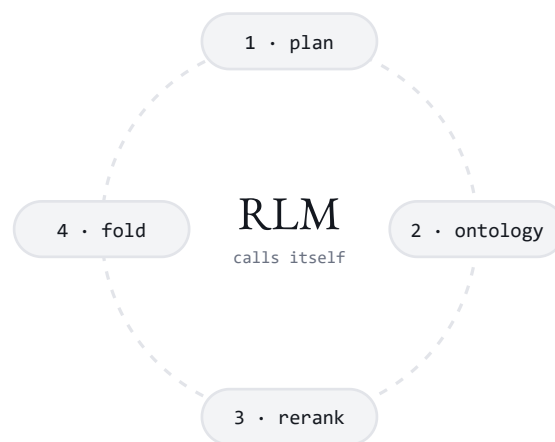


OPEN SOURCE · WHITELABEL **COMING SOON**

The self-learning context layer for funds.

The highly versatile context layer for professional fund controlling. Built for demanding agentic loops.



LOOP TRACE

```

> plan()
  fee · Fund Alpha · valid today

> ontology.query(subject="fund_alpha")
  3 entities · 11 events · 214 bookings

> rerank(top_k=200, model="ctx-r1")
  12 passages out of 214,000

> sub_llm × 8 → fold()
  482,000 tokens read · 1,400 kept

✓ answer.ready = True
  "1.5% p.a." · LPA 2024 § 6.2
  
```

ONE PASS

482,000 tokens read

prompt era		context rot
RLM loop		flat

Same work, same question. One context fills up and loses its middle. The other stays flat — and can keep going.

The prompting era is over.

For three years the answer to every problem was a better prompt. Documents were pasted into a context window, an instruction written on top, and then came hope. That works while the questions stay small. In a fund they never are: a question about a management fee touches the partnership agreement, three amendments, two side letters, eighteen account sheets — and the question of which version holds today.

Context windows grew, and the problem grew with them. Cost rises linearly with length while accuracy falls. Context rot is the name for it: a model that reads a million tokens remembers the beginning and the end and loses the middle. The middle is where the superseded version sits.

What comes instead is a loop. The model is no longer handed the data inside its context — it is handed the data itself: it queries the ontology, calls the reranker, reads the hits, folds the rest back away, and starts again until the answer holds.

Recursive means nothing mystical here. The model starts fresh copies of itself and sends them off to read what it does not want in its own head: eight account sheets, a set of agreements, a four-hundred-page PDF. What comes back is not the pages but the result. The model calls itself — hence recursive. And because its own context stays short, it can keep working long after a single-prompt system has filled up.

Not more context into the model. A place for the model to go, where the context already lies in order.

THE LOOP

What a loop is — and what managing one means.

A loop is not a chat. An agent in a loop runs the same work over and over — work, check, correct, continue — until a stop condition is met. Nobody types between the passes. That moves where the real work sits: it is no longer writing the next prompt, it is building the loop. What triggers it, what it checks itself against, when it stops.

A prompt is a sentence. A loop is a system. And loops differ above all in what gets handed over.

FOUR SHAPES

Turn-based

HANDED OVER

the check

TRIGGERED BY

a task someone sets

ENDS

when the agent judges the work done

SUITED TO

short, open-ended tasks with someone reading along

Goal-based

HANDED OVER

the stop condition

TRIGGERED BY

the previous pass finishing

ENDS

when an independent check confirms the goal — or the pass limit bites

SUITED TO

tasks with a verifiable “done”

Time-based

HANDED OVER

the trigger

TRIGGERED BY

an interval

ENDS

when cancelled, or when the watched work finishes

SUITED TO

recurring work, waiting on external state

Proactive

HANDED OVER

the task itself

TRIGGERED BY

an event or a schedule

ENDS

when operations switch it off

SUITED TO

standing streams: inbox, reconciliation, migration

Loop management is the discipline that follows, and at its core it comes down to three questions.

The first question is the hard one — and it is why this layer exists. A loop may only run unattended if it can check its own result. In a fund, checked means nothing vague: the booking has two legs and they close. The amount appears in a document, with a page and a paragraph. The version it came from is the one that holds today. The occurrence is complete — no capital call without its counter-booking, no valuation without a period.

First: how does the loop know its work is right? Without a reliable check, a loop is just a machine repeating its own mistakes — quickly and tirelessly. Second: what stops it? Stop conditions, pass limits, budgets, checkpoints; without them it runs until the budget is gone. Third: what does it draw on? Codified domain knowledge instead of re-explaining on every run, separate working areas for parallel agents, sub-agents for bounded pieces of work, and state that survives between passes.

Those are exactly the questions an ontology can answer and a pile of documents cannot. So the context layer is not merely the loop's data source. It is the loop's verifier — and therefore the condition on which running unattended rests at all.

One layer beneath every function.

Fund operations break into functions: capital calls, valuation, reconciliation, reporting, tax, compliance. Each has historically kept its own filing, its own truth, its own spreadsheet. The context layer runs underneath all of them — four layers deep, the same ground for every function. It is not a chatbot and not a search box; it is the floor both of those stand on.

Fund operations

IN THE OPERATOR'S OWN APPLICATION

Capital calls

NAV & valuation

Cash & reconciliation

LP reporting

Tax & close

Compliance & audit

The context layer

ONE LAYER, RUNNING ACROSS EVERY FUNCTION

01 Context loop

LOOP

The agent gets no prompt — it gets a loop: plan, query, rerank, fold.

plan ontology.query rerank sub_llm fold

02 Retrieval

PRECISION

The first stage casts wide; a cross-encoder trained on the holdings decides what comes out on top.

trigram BM25 vector cross-encoder on own data

03 Ontology

MEANING

Documents become sentences with a subject and an object — a ledger, not a pile of chunks.

entities events bookings measurements citations

04 Intake

SOURCE

Everything arriving becomes a document with a page, a line and a timestamp.

inbox

portals

banks

folder sync

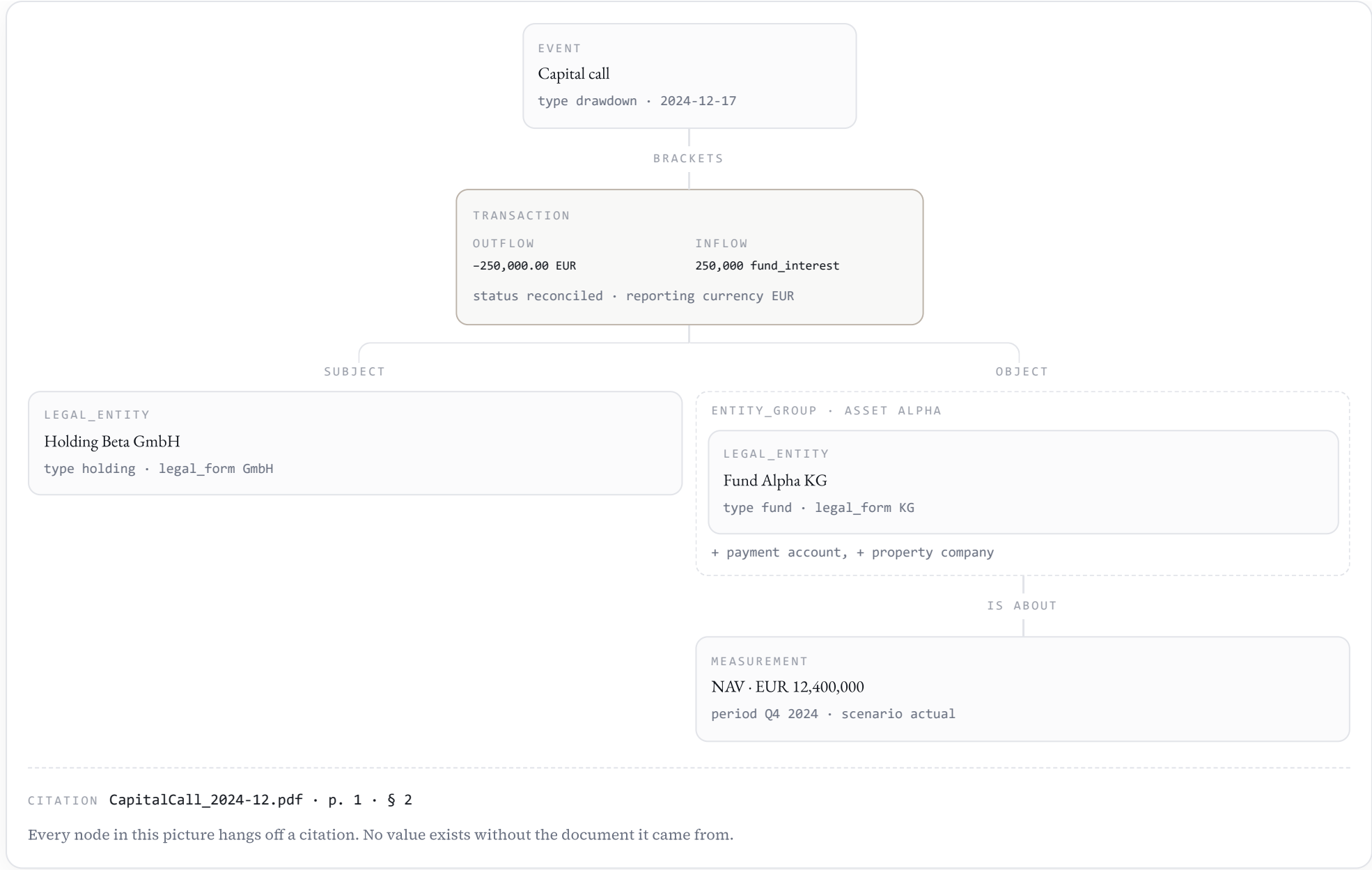
OCR

No function gets its own filing cabinet. They all reach for the same data, the same citations, the same validity.

A ledger, not a pile of chunks.

Every RAG prototype of the last few years ended in the same place: cut documents into pieces, turn pieces into vectors — and then discover that “€250,000” has no direction in a vector space. An amount without a subject, without a counterparty, without a date is not information. It is a number on paper.

That is why the second layer is not a vector database but a ledger. Every statement gets a grammar: who, against whom, when, how much — out and back, because every booking has two legs. Above that sit occurrences that bracket the bookings belonging together, groups that turn several companies into one asset, and measurements with period and scenario. And underneath everything, the citation stays.



Five kinds of sentence

legal_entity

WHO AND WHAT

Funds, companies, people, accounts, assets. Anything that can be a subject or a counterparty.

entity_group

WHAT BELONGS TOGETHER

An asset is rarely one entity. The group bundles property company, account and stake into a single thing you can talk about.

event

THE OCCASION

The drawdown, the valuation, the distribution. It brackets the bookings that belong to one occurrence.

transaction

WHAT MOVED

Two legs — outflow and inflow. The sign is the direction, the reporting currency always travels along.

measurement

WHERE IT STANDS

NAV, revenue, EBITDA, fair value — with period, scenario and instrument, not as free text.

This is the part that cannot be bought off the shelf and cannot be generic. A family office books differently from a fund of funds, a KG differently from a holding. The ontology is shaped to the firm it serves: same grammar, its own vocabulary.

And because it is a grammar, it is queryable. An agent does not need to read 214,000 passages to know what Fund Alpha drew down in 2024. It asks the ledger a question and gets 214 rows. That is the difference between a model that searches and a model that knows.

A reranker that speaks the language of the house.

The first stage of any retrieval is optimised for recall: BM25 for exact terms, trigram for spelling variants and OCR noise, vector search for paraphrases. It returns broadly — and in rough order, because none of these methods understands the question.

The second stage decides. A cross-encoder reads question and passage together; every word of the question sees every word of the passage, and out comes a calibrated score. Off-the-shelf models are trained on multilingual web corpora, which is doubly wrong for German financial language. Kapitalerhöhungsbeschluss shatters in the tokenizer, Kapitalkonto II sits outside the training distribution — and a web model cannot see that the 2021 version is superseded: it is nearly token-identical to its successor.

So one is trained on the holdings themselves: the agreements, account sheets and side letters of the firm. Training pairs come from the ontology, hard negatives from superseded versions and twin passages — exactly the cases where generic models rank plausibly and wrongly. Small enough for the hot path: single-digit milliseconds, on every typed word.

In practice nobody reads ten results. What gets read is the first. Whether a system earns trust or squanders it is decided right there.

And it does not stand still.

Every pass of a loop leaves signal behind. Which passage carried the answer and which one missed. Which version turned out to be the one that holds. Where a correction was needed and what it hinged on. Where a booking failed to close and which document finally closed it.

A loop runs

→

Cited hits and corrections become signal

→

Signal becomes training pairs

→

The next reranker is trained

↻ and the sharper reranker produces cleaner signal again

That signal becomes training pairs: the cited hits as positives, the near-miss twins as hard negatives. The next generation of the reranker is trained on them — and it improves precisely where the last one reached wrong. Retrieval stops being a bought-in component and becomes a mechanism that sharpens itself.

So the circle closes, and it closes inward: the loop queries the retrieval, the retrieval improves from the loop, the better loop produces cleaner signal. What grows out of that is not a product off a shelf but a framework of its own — data, loops and retrieval mechanics, grown in one place and only that effective there.

Top-1

the only position that matters in daily work

< 10 ms

reranking inside the latency budget of search

Own weights

trained on the holdings of the house, not on the web

What gets tailored, and what comes with it.

The source is open, the implementation is cut to fit the firm it serves, and what steps forward is the operator's own application. The client relationship stays where it belongs. What should sit underneath it is the best data layer in the German-speaking fund market.

Tailored

- Brand, domain, interface
- Workflows and approvals
- Ontology and vocabulary
- The reranker's model weights
- Where it runs — own cloud or ours

Included

- Intake, conversion, extraction
- Index, embeddings, reranking
- The context loop and its API
- Operations, monitoring, SLA
- The research behind it

One set of holdings trains one model and no other. No shared embeddings, no shared weights, no training across anyone else's holdings.

Whoever holds the context layer holds the answer.

The models become interchangeable. They get better, cheaper and more alike every year, and in five years nobody will remember which one was picked in 2026.

What does not become interchangeable is the framework beneath: ordered, cited, queryable data; loops that can check themselves against that data and may therefore run alone; and a retrieval that sharpens with every pass. The three hold each other up — each part makes the others better.

Something like that is built once, it is built where the documents are, and it cannot be bought. That is the moat.

Get in touch

The context layer is being built. For firms considering an implementation, an early conversation pays — the ontology takes shape with the documents, not after them.

cd@fundops.de

[About / Imprint](#) [Privacy](#)